

PYADV corrigé des exercices

** permettant l'emballage de dictionnaires, l'exécution du code suivant :

```
def get_positional_args(**kwargs):  
    print(kwargs)  
  
get_positional_args("Hello", "World")
```

Produit naturellement une erreur très parlante :

```
TypeError: get_positional_args() takes 0 positional arguments but 2 were  
given
```

Mais si l'on exécute la fonction avec des arguments nommés on constate que ceux-ci sont parfaitement récupérés par la fonction :

```
get_positional_args(hello="Hello", world="World")
```

Affiche :

```
{'hello': 'Hello', 'world': 'World'}
```

De manière identique, le code suivant produit une erreur :

```
def get_named_args(*args):  
    print(kwargs)  
  
get_named_args(hello="Hello", world="World")
```

```
TypeError: get_named_args() got an unexpected keyword argument 'hello'
```

Des résultats que vous avez obtenus, définissez une fonction pouvant récupérer des arguments positionnels et des arguments nommés.

Créez un context manager qui délimite le code exécuté avec un nombre de caractères défini par ses paramètres.

Écrivez un programme qui analyse une liste et produit 2 nouvelles listes contenant les nombres pairs pour l'une et impairs pour l'autre.

```
# Create a list of 20 numbers
numbers = list(range(20))

odd = [i for i in numbers if i % 2 != 0]
even = [i for i in numbers if i % 2 == 0]

# Or like this:
#odd = [i for i in numbers if i % 2]
#even = [i for i in numbers if not i % 2]

print("Impair (odd):", odd)
print("Pair (even) :", even)
```

En utilisant le principe du Crible d'Eratosthène affichez les nombres premiers plus petits que 100.

L'approche du cours PYTHON utilisait deux boucles imbriquées afin de **retirer** les nombres non premiers.

Avec une approche utilisant les *list comprehension* il s'agit de **créer** une nouvelle liste.

Le problème est plus complexe car si un nombre n'est pas divisible par un autre (imbrication des boucles) il sera automatiquement ajouté à la liste.

L'approche consiste donc à créer une liste de tous les multiples et d'en faire la différence avec la liste d'origine.

Et cela en utilisant un *set comprehension* :

```
numbers = range(2, 101)

sieve = set(numbers).difference({y for x in numbers for y in numbers if y > x
and y % x == 0})

print(sieve)
```

Créez un générateur qui inverse un itérable (string, list, ...) qui lui est passée en argument.

Testez avec des chaînes de caractères pour commencer.

Le générateur peut être défini par la fonction suivante :

```
def reverter(iterable):
    # Convert the argument to a list to ensure it can be reverted
    iterable = list(iterable)
```

```
# Revert it
iterable.reverse()

# Yield each element in order
for i in iterable:
    yield i
```

Ou plus simplement :

```
def reverter(iterable):
    length = len(it)

    # Yield each element in reverse order
    for i in range(length - 1, -1, -1):
        yield it[i]
```

Que l'on peut tester avec :

```
s = ''.join(reverter("Hello World"))

l = list(range(20))
l = list(reverter(l))
```

Créez un générateur qui retourne la puissance N des premiers X nombres entiers.

X est passé au moment de la création du générateur.

N vaut $1/2$ par défaut, mais peut être modifiée en cours d'utilisation.

Le code du générateur pourrait être le suivant :

```
def power(max = 0):
    cur = 0
    power = 1/2
    while cur <= max:
        message = (yield cur ** power)
        if message is not None:
            power = message
        else:
            cur += 1
```

Et il pourrait être testé de cette manière :

```

print("Square root of numbers 0 to 3:")
for i in power(3):
    print(i)
print()

i = power(4)
print("Square root of 0 and 1:")
print(next(i))
print(next(i))
print()

# Change power value to 2
i.send(2)
print("Power 2 of 2:")
print(next(i))
print()

# Change power value to 5
i.send(5)
print("Power 5 of 3 and 4:")
for val in i:
    print(val)

```

Soit le fichier de log d'une chaîne de pizza (`ventes.log`).

La 4ème colonne de chaque ligne affiche le nombre de pizzas vendues chaque heure ou N/A si aucune pizza n'a été vendue.

Vous vous intéressez au total des pizzas vendues ces 10 dernières années.

Considérez que le fichier contient exactement les 10 dernières années.

L'idée consiste à chaîner des générateurs afin de limiter l'utilisation mémoire.

```

import csv

with open('ventes.log') as fd:

    reader = csv.reader(fd,
                        delimiter=',',
                        quotechar='"',
                        quoting=csv.QUOTE_NONNUMERIC)

    # A generator to get the 4th field of each lines

```

```

pizza_col = (line[3] for line in reader if line)

# The previous generator is used by this one which get the number
# of sold pizza only if there was sold pizza during this hour
per_hour = (int(x) for x in pizza_col if x != 'N/A')

# Finally the sum function will sum each value returned by the
# chained generators
print("Total pizzas sold =", sum(per_hour))

```

Écrivez un programme qui demande à l'utilisateur d'entrer un nombre (entier ou réel).

Le programme vérifie la validité de l'entrée à l'aide d'une expression régulière.

Nombres valides: 8, 888888.5, 0.5, -8

Nombres invalides: 8., .5, 8.5.5, - 8

La méthode `match` du module `re` permet de vérifier si une string correspond à un pattern en effectuant le *matching* depuis le début de la string, ne nécessitant ainsi pas l'utilisation de l'assertion `^` :

```

import re

while True:

    number = input("Enter a number (q to quit): ")

    if number.lower() == 'q':
        break

    # match automatically start from the begining of the string, so we don't
    # have to use ^
    if re.match("-?\d+(\.\d+)?$", number):
        print(f"Ok, {number} is a real number")
    else:
        print(f"FAILED! {number} doesn't represent a valid number. Try
again.")

```

Ajoutez le support du logging à l'exercice précédent afin que les entrées invalides soient logguées dans un fichier avec l'heure à laquelle elles ont été entrées.

En utilisant et configurant le module `logging` de manière adéquate à l'aide de la méthode `basicConfig` on obtient exactement ce qui est demandé.

Dans la configuration on spécifie le `format` afin de définir les informations enregistrées dans le fichier de

log, ainsi que le format de sauvegarde des informations de date et heure avec le paramètre `datefmt` (pour ne pas avoir de micro-secondes).

```
import re
import logging

logging.basicConfig(filename="debug.log",
                    format="%(asctime)s %(levelname)s: %(message)s",
                    datefmt="%F %T",
                    level=logging.WARNING)

while True:

    [...]

    else:
        print(f"FAILED! {number} doesn't represent a valid number. Try
again.")
        logging.error(f"{number} is not valid")
```

La facture du restaurant se monte à 100. – et vous avez sur vous 1 billet de 100, 3 de 50, 4 de 20 et 4 de 10.

Quelles sont vos possibilités de régler ce montant ?

La fonction `combinations` du module `itertools` permet de gérer les combinaisons non redondantes de `X` parmi `Y` éléments.

Un `set` permet de s'assurer de l'unicité des éléments.

Un `set` ne peut cependant contenir que des éléments *hashable*, ce qui est le cas des `tuple` retournés par la fonction `combinations`.

Finalement il est possible de trier une `list` en fonction d'une clé de tri représentée par une fonction associée au paramètre `key`.

```
from itertools import combinations

# Create the banknotes list
banknotes = [100] + [50] * 3 + [20] * 4 + [10] * 4

# Save all in a set to automatically remove duplicates
possibilities = set()

# Possible combinations are made of X number of banknote(s)
```

```

# Where X range from 1 to the total number of banknotes
for i in range(1, len(banknotes) + 1):
    for c in combinations(banknotes, i):
        if sum(c) == 100:
            possibilities.add(c)

# Transform the set in a list to be able to sort results
possibilities = list(possibilities)

# Sort results by number of banknotes descending
possibilities.sort(key=len, reverse=True)

# Print results
for p in possibilities:
    print(p)

```

Déplacer les fonctions de l'exercice 4 dans un module et utilisez-les depuis un autre programme.
Tester l'importation de différentes manières.

Les fonctions `reverter` et `power` sont déplacées dans un fichier `functions.py`, ce qui crée un module `functions`.

Depuis le programme principal dont le code se trouve dans un autre fichier, l'utilisation des fonctions est effectué suivant l'une des deux approches suivante :

```

import functions

if __name__ == "__main__":
    s = ''.join(functions.reverter("Hello World"))
    print(s)

    for i in functions.power(3):
        print(i)

```

```

from functions import reverter, power

if __name__ == "__main__":
    s = ''.join(reverter("Hello World"))
    print(s)

    for i in power(3):
        print(i)

```

Créez une classe permettant de représenter un animal familial (`class Pet`) qui a les caractéristiques et les comportements suivants :

Il a un âge, un poids, un nom.

Il peut manger une quantité de "pâtée pour pets", ce qui le fera grossir de 20% de la quantité ingurgitée.

Il peut également dormir et parler.

Lorsqu'on le fait parler, il est possible de lui passer en paramètre ce qu'il doit dire.

Ajoutez un compteur automatique de manière à savoir en tout temps le nombre de Pets créés.

Créez des classes dérivées pour définir des animaux particuliers (`Dog`, `Cat`, `Fish`).

L'intérêt étant de pouvoir écraser des fonctions de la classe de base dans les classes dérivées.

Vous devriez maintenant être en mesure de faire aboyer le chien et miauler le chat.

En considérant que la définition de chacune des classes se trouvent dans le fichier `pet.py` il ne sera pas nécessaire d'importer la définition de la classe `Pet`.

Dans un cas réel, chaque classe serait définie dans son propre fichier, `pet.py`, `cat.py` et `dog.py`.

```
# class Pet:
# [...]

class Dog(Pet):
    def __init__(self, name, age, weight):
        """Due to parametrized constructor"""
        super().__init__(name, age, weight)

    def speak(self, msg=""):
        """Override speak method to prevent Dogs being able to speak"""
        print(f"{self.name}: Warf warf!")

class Cat(Pet):
    def __init__(self, name, age, weight):
        """Due to parametrized constructor"""
        super().__init__(name, age, weight)

    def speak(self, msg=""):
        """Override speak method to prevent Cats being able to speak"""
        print(f"{self.name}: Meeooooow")
```

`*args` permet donc de récupérer les arguments positionnels et `**kwargs` permet de récupérer les arguments nommés.

Comme les arguments nommés doivent être récupérés après les arguments positionnels, la fonction suivante devrait permettre de récupérer tant les arguments nommés que positionnels qui lui sont transmis :

```
def get_all_args(*args, **kwargs):
    print(args)
    print(kwargs)
```

Ce qui est bien le cas si on la teste avec les 4 différentes possibilités :

```
get_all_args()
# ()
# {}
```

```
get_all_args("Hello", "World")
# ('Hello', 'World')
# {}
```

```
get_all_args(hello="Hello", world="World")
# ()
# {'hello': 'Hello', 'world': 'World'}
```

```
get_all_args("Hello", world="World")
# ('Hello',)
# {'world': 'World'}
```

Aucune erreur n'est générée et les résultats sont bien ceux auxquels on s'attend

Il serait possible de mettre des valeurs par défaut pour les paramètres du constructeur, par exemple :

```
def __init__(self, name="NONAME", age=0, weight=0):
```

Mais cela signifie que l'on peut créer un `Pet` par vraiment réaliste.

En ne mettant pas de valeurs par défaut on force la création d'un `Pet` plus *réfléchi*.

Le fichier `pet.py` pourrait contenir le code suivant :

```
class Pet:
    counter = 0

    def __init__(self, name, age, weight):
        self.name = name
        self.age = age
        self.weight = weight
```

```

        Pet.counter += 1
        print(f"{self.name}: Hello World, I'm ALIVE")
        print(f"{self.name}: We are now {Pet.counter}")

    def __del__(self):
        Pet.counter -= 1
        print(f"{self.name}: Bye bye...")
        print(f"{self.name}: We are one less, that is to say only
{Pet.counter}")

    def speak(self, msg=""):
        print(f"{self.name}: {msg}")

    def eat(self, quantity):
        self.weight += quantity * 0.2
        print(f"{self.name}: I'm now {self.weight:.2f}kg fat")

    def sleep(self):
        print(f"{self.name}: ZZZZzzzzzzzz")

```

```

import functools

def my_decorator(char, nb):

    print(f"Decored with {nb} {char}")

    def decorator(function):

        @functools.wraps(function)
        def inner(*args, **kwargs):
            print(char * nb)
            function(*args, **kwargs)
            print(char * nb)

        return inner

    return decorator

@my_decorator('*', 20)
def hello(who, times):

```

```
"""Say hello multiple times to somebody"""
for i in range(times):
    print(f"Hello {who}!")

hello("Julien", times=3)
```

Un context manager est défini par la présence des méthodes `__enter__` et `__exit__`.

La classe suivante permet de réaliser le résultat demandé :

```
class CodeDelimiter:
    def __init__(self, char, length):
        self.char = char
        self.length = length

    def __enter__(self):
        print(self.char * self.length)

    def __exit__(self, ex_type, ex_value, ex_traceback):
        print(self.char * self.length)
```

Par exemple, pour éviter que le nom d'un `Pet` soit plus long que 20 caractères, on définit ce nom en tant que propriété en ajoutant le code suivant dans la classe `Pet` :

```
@property
def name(self):
    return self.__name

@name.setter
def name(self, name):
    name = str(name)
    if len(name) > 20:
        raise Exception("Max name length: 20 chars")
    self.__name = name
```

Ce qui oblige également à modifier, dans la classe, les appels à `self.name` par `self.__name`.

En particulier dans le constructeur :

```
def __init__(self, name, age, weight):
    self.__name = name
```

```
# ...
```

Que l'on pourra tester, par exemple, avec :

```
tobby = Pet("Tobby", 2, 6.5)

print(f"tobby name: {tobby.name}")
try:
    toby.name = "Ils ont des chapeaux ronds, vive les bretons"
except Exception as e:
    print(e)

print(f"tobby name: {tobby.name}")
```

Sauvegardez vos animaux dans un tableau et faites-les parler à l'aide d'une boucle.

A nouveau, par simplification, considérons que tout le code se trouve dans un même fichier.

```
# class Pet: [...]
# class Dog(Pet): [...]
# class Cat(Pet): [...]

if __name__ == "__main__":

    my_pets = [
        Dog("Tobby", 2, 6.5),
        Cat("Felix", 3, 4)
    ]

    for pet in my_pets:
        # They should just bark and meow
        pet.speak("Hello World!")
```

Le chat miaule et le chien aboie !

Modifiez votre code de l'exercice 5 afin d'utiliser une approche plus *pythonique*, basée sur les exceptions.

Utilisez une approche EAFP plutôt que LBYL.

Les entrées `.8` et `8.` seront cette fois considérées comme valides.

```
while True:

    number = input("Enter a number (q to quit): ")
```

```

if number.lower() == 'q':
    break

# Try to convert the number to a float
# If conversion fails, the input was not representing a number
try:
    res = float(number)
    print(f"Ok, {number} is a real number: {res}")
except:
    print(f"FAILED! {number} doesn't represent a valid number. Try
again.")

```

L'importation du module `re` n'est plus nécessaire.

Le code est plus simple et plus compréhensible.

Écrivez un décorateur qui empêche l'exécution d'une fonction lorsque les secondes sont impaires.

L'exécution de la fonction affichera l'heure courante et :

- Dans le cas de secondes paires, l'output de la fonction.
- Dans le cas de secondes impaires, un message explicatif.

```

import time

def my_decorator(function):

    def inner():
        now = time.localtime()
        if time.localtime().tm_sec % 2:
            print(f"{time.strftime('%T', now)}: Odd seconds prevent
execution")
        else:
            print(f"{time.strftime('%T', now)}:", end=" ")
            function()

    return inner

@my_decorator
def hello():
    print("Hello World!")

for i in range(10):

```

```
hello()
time.sleep(0.75)
```

Écrivez un décorateur qui délimite la sortie produite par une fonction avec des lignes constituées d'un nombre variable de caractères spéciaux.

Les paramètres du décorateur sont le nombre et le caractère à utiliser pour créer les lignes. La sortie pourrait être du style :

```
*****
Hello world !
*****
```

```
def my_decorator(char, nb):

    def decorator(function):

        def inner():
            print(char * nb)
            function()
            print(char * nb)

        return inner

    return decorator

@my_decorator('*', 20)
def hello():
    print("Hello World!")

hello()
```

Modifiez le décorateur de l'exercice précédent afin de pouvoir utiliser une fonction prenant en paramètre le texte à afficher.

Dans le décorateur, affichez les paramètres reçus pour la fonction décorée.

Écrivez un décorateur qui permette de tracer le temps d'exécution de n'importe quelle fonction.

L'output de la fonction doit évidemment pouvoir être récupéré lors de son exécution.

Testez avec une fonction permettant de suspendre l'exécution du programme, telle que `input()` ou `time.sleep()`.

```

import time
import functools

def time_tracer():

    def decorator(function):

        @functools.wraps(function)
        def inner(*args, **kwargs):
            start = time.time()
            function(*args, **kwargs)
            print(f"Execution time: {time.time() - start}")

        return inner

    return decorator

@time_tracer()
def slow_count(start=0, end=9, delay=0):
    for i in range(end + 1):
        time.sleep(delay)
        print(i, end=' ', flush=True)
    print()

slow_count(delay=0.5)

```

Faites le nécessaire afin de pouvoir afficher facilement les informations d'un `Pet`.

Il suffit d'ajouter, dans la classe `Pet`, la méthode *dunder* `__str__()` qui sera automatiquement appelée lors de l'affichage d'un objet de la classe.

```

def __str__(self):
    return f"{self.name}, {self.age}, {self.weight}"

```

Ajoutez à votre programme une méthode permettant de récupérer le nombre de `Pet` créés.

Pour éviter les accès externes, la variable `counter` devient privée => `__counter`. Ce qui implique quelques modifications dans le code de la classe.

Pour récupérer le nombre de `Pet` créés on ajoute une méthode :

```
def get_number_of_pets():
    return Pet.__counter
```

Encapsulez les informations importantes (nom, poids, âge) de votre application en utilisant des propriétés.

Créez une classe `Currency` permettant de manipuler des sommes monétaires dans différentes monnaies et qui pourrait être utilisée de la manière suivante :

```
from currencies import Currency
v1 = Currency(20.50)
v2 = Currency(20.50, "EUR")
v3 = Currency(20.50, "USD")
print(v1 + v2)
# => 43.28 CHF
print(v2 + v1)
# => 38.95 EUR
print(v3 + 5)
# => 25.50 USD
print(5 + v3)
# => 25.50 USD
```

```
class Currency:

    change = {
        'CHF': 1.0,
        'EUR': 0.9,
        'USD': 1.1
    }

    def __init__(self, value, unit="CHF"):
        self.value = value
        self.unit = unit

    def __str__(self):
        return f"{self.value:5.2f} {self.unit}"

    def __add__(self, other):
        if type(other) == int or type(other) == float:
            x = other
        else:
            x = (other.value / Currency.change[other.unit] *
Currency.change[self.unit])
```

```

        return Currency(x + self.value, self.unit)

    def __radd__(self, other):
        res = self + other
        return res

if __name__ == "__main__":

    v1 = Currency(20.50)
    v2 = Currency(20.50, "EUR")
    v3 = Currency(20.50, "USD")

    print(v1 + v2)
    print(v2 + v1)
    print(v3 + 5)
    print(5 + v3)

```

Créez une interface à votre programme de l'exercice 8 afin de permettre à l'utilisateur d'avoir une liste de `Pet` et de demander des informations à propos d'une caractéristique que le `Pet` pourrait avoir.

Par exemple pour savoir si le `Pet` a un âge est savoir quel est cet âge, l'utilisateur écrirait simplement `age` sur la ligne de commande de l'interface du programme.

Une interface simple pourrait être la suivante :

```

if __name__ == "__main__":

    pet_list = [
        Dog("Tobby", 2, 6.5),
        Cat("Felix", 4, 4),
        Dog("Snoopy", 5, 6)
    ]

    while True:

        print()
        print("[L]ist all pets")
        print("[X] Select pet #X")
        print("[Q]uit")

        what = input("? ").lower()
        if what == 'q':

```

```

        break

if what == 'l':
    for i in range(len(pet_list)):
        print(f"{i:2d} {pet_list[i].__class__.__name__}")
    continue

# Here *what* should be the index of the pet in the list
# So convert it to an int
try:
    what = int(what)
    if what >= len(pet_list):
        continue
except:
    continue

pet = pet_list[what]

while True:
    prop = input("What property are you interested in ? ")
    if not prop:
        break

    if hasattr(pet, prop):
        print(f"Pet #{what} '{prop}': {getattr(pet, prop)}")
    else:
        print(f"Pet #{what} has no '{prop}'")

```